Heart Risk: Reproducible Classification

AIMLCZG523 | Assignment 01 | End-to-end MLOps

Author: Ayush Kumar, 2025AE05258

Repository: GitHub Heart Disease Prediction

Video: Open MLOps_Recording.mp4 (keep the video beside this PDF for offline access)

This report summarizes an end-to-end machine-learning workflow using the UCI Cleveland heart-disease dataset. It covers data preparation, model evaluation, experiment tracking, API serving, automated checks, and local deployment.

Logistic Regression and Random Forest were evaluated with the same data split and cross-validation approach. Preprocessing is fit within training folds. Model selection uses validation ROC-AUC, while the holdout set is reserved for final evaluation. The close validation results do not establish statistical superiority.

The selected pipeline is served through FastAPI on Docker Desktop Kubernetes. MLflow records experiments, and GitHub Actions checks code, tests, training, and container delivery. The hosted workflow publishes a tested image; local Kubernetes deployment is run separately.

This is an educational classifier, not a clinically validated device. It predicts the dataset's recorded disease label, not future cardiovascular risk. Model probabilities are not medical advice or guarantees.

Selected model	CV ROC-AUC	Holdout recall
logistic_regression	0.911	0.929

Architecture and Reproduction

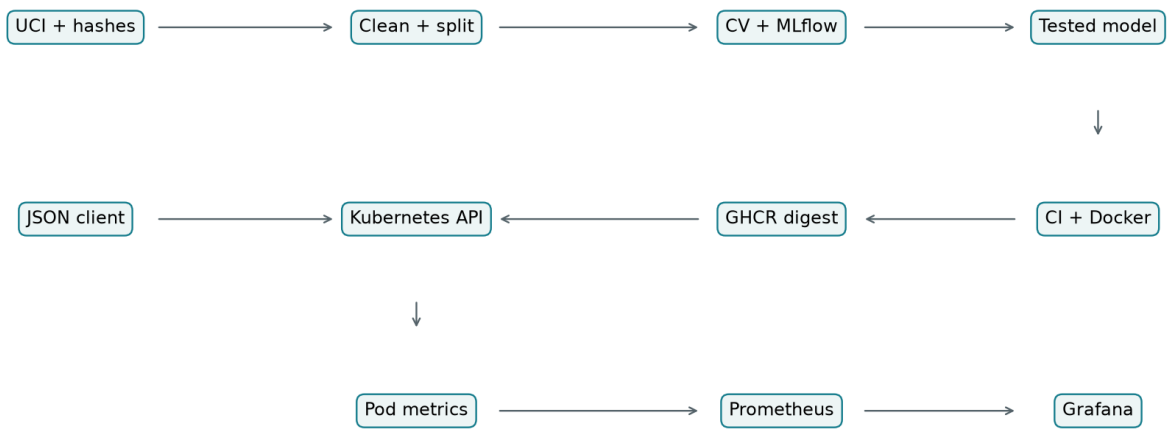
Data lineage, artifact boundaries, and environment isolation

The official source snapshot is validated before cleaning and splitting. Training produces a saved preprocessing-and-model pipeline with metadata and evaluation results. The API loads that trusted artifact at startup; inference does not train a new model or download data.

Python 3.11 and locked dependencies are used across development, CI, and the serving container. Separate full and serving requirements keep the training and runtime environments reproducible.

The README documents setup and the reproduction sequence: quality checks, data preparation, EDA, training, inference, and API verification. The PowerShell runner stops when a command fails.

MLflow uses local storage for this assignment. Portable results and model artifacts accompany the tracking data; the local tracking database is not presented as a shared production service.



Original architecture: tested artifacts flow toward serving; telemetry flows separately.

Data Provenance and Cleaning

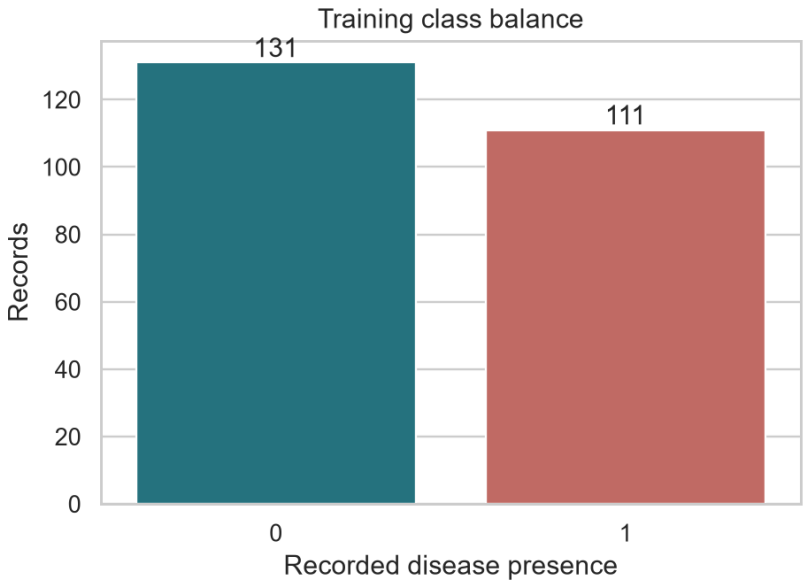
Official Cleveland subset; no third-party remapping

The project uses the official UCI Cleveland subset: 303 records, 13 predictors, and a disease-severity target. The assignment refers to 14 or more features; the canonical source has 13 predictors plus the target. No artificial predictors are added.

Severity is mapped to a binary disease-presence label. The source's original category codes are preserved and validated during both ingestion and API requests.

The cleaning process validates rows and values, converts missing markers to nulls, and records data provenance. Missing feature values remain in the cleaned dataset until model preprocessing, avoiding information leakage.

A reproducible stratified split and checksums identify the data used for analysis. The dataset source and attribution are documented in the repository.



Class balance for the training partition; the source classes are reasonably balanced.

EDA and Feature Processing

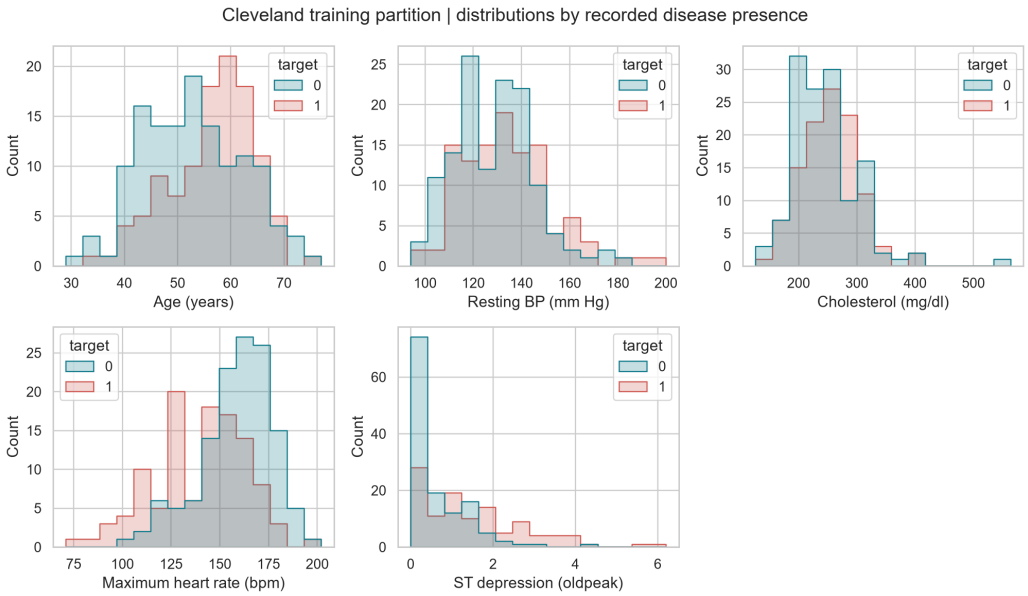
Training-only interpretation and fold-local transformations

Exploratory analysis is performed on the training partition and summarizes feature distributions, class balance, missingness, and relationships among selected variables. The plots are descriptive, not evidence of causal effects.

The model pipeline handles numeric and categorical features with suitable preprocessing. Categorical codes are treated as categories rather than ordered measurements.

Preprocessing and classification are combined in a single pipeline and fit within each cross-validation fold. This keeps validation data from influencing learned preprocessing and supports consistent inference.

The initial comparison uses the source predictors without target-derived features. The same documented feature contract is enforced by the API.



Distributions generated from the fixed training set; missingness and correlation figures accompany the repository.

Model Selection and Evaluation

Shared folds, bounded searches, and a locked decision threshold

The comparison evaluates Logistic Regression and Random Forest using the same stratified cross-validation design and bounded hyperparameter searches. Both models use the same training data and fold-local preprocessing.

Validation ROC-AUC is the selection measure. Accuracy, precision, recall, F1, and ROC-AUC summarize model performance, with cross-validation variability reported alongside the averages.

The model choice is fixed before the holdout set is used. A consistent decision threshold is applied to both models, and holdout results are not used to retune the pipeline.

Results on this small historical dataset are exploratory, not evidence of general clinical performance. Further validation on representative populations would be needed before any clinical use.

Metric	Logistic Regression	Random Forest
Holdout accuracy	0.885	0.869
Holdout precision	0.839	0.812
Holdout recall	0.929	0.929
Holdout f1	0.881	0.867
Holdout roc_auc	0.965	0.953
CV ROC-AUC mean (SD)	0.911 (0.030)	0.909 (0.035)

Experiment Tracking and Packaging

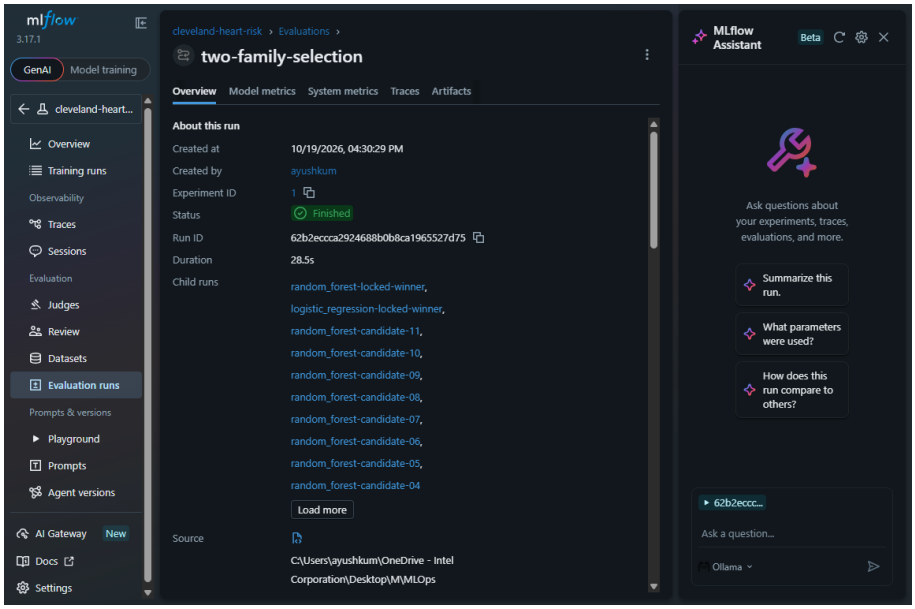
Inspectable MLflow runs and trusted model persistence

MLflow organizes the experiment into an overall run and related model-search runs. It records configurations, evaluation metrics, and artifacts so the comparison can be inspected and reproduced.

The saved serving bundle contains both preprocessing and the selected classifier, along with metadata describing the model and expected input schema. Inference checks the bundle before loading it; serialized models must come from a trusted source.

The project verifies that the saved model behaves consistently with the fitted pipeline. The container includes the selected artifact and does not train a replacement at startup.

This packaging approach keeps experiment evaluation and API inference connected while allowing each part to run in its intended environment.



Local MLflow experiment showing the tracked run and linked model-search results.

Automated Tests and CI/CD

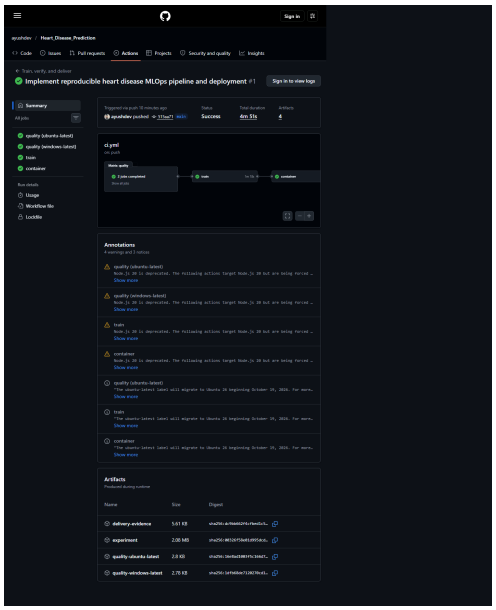
Failure propagation and delivery of the same tested image

Unit tests cover the data contract, preprocessing, model behavior, API validation, and delivery configuration. A separate integration workflow exercises the complete training and serving path.

GitHub Actions runs quality checks on Windows and Ubuntu before training on Linux. The resulting model is packaged into a container and smoke-tested in isolation.

On trusted pushes, the tested image is published to GitHub Container Registry and identified by its immutable digest. Pull requests run validation without publishing an image.

Required checks fail the workflow when code, tests, training, or container verification fails. Logs and artifacts are retained to support diagnosis and reproducibility.



Successful hosted workflow showing cross-platform checks, training, container verification, and image publication.

API and Container Isolation

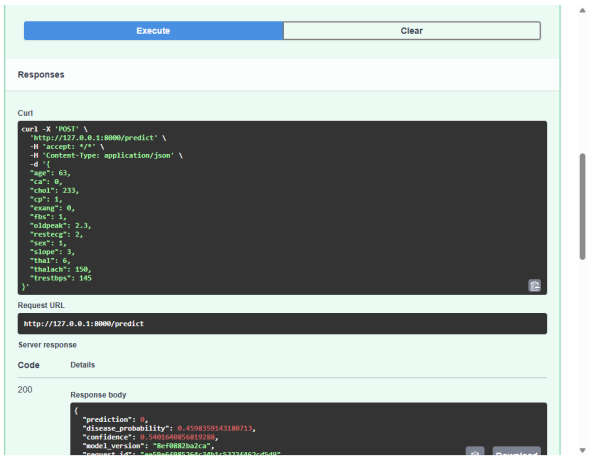
Validated requests, deterministic inference, and runtime boundaries

The API accepts a patient record that follows the documented feature schema. It validates required fields, data types, and category codes, returning a client error for unsupported input.

Responses provide a binary prediction, disease probability, confidence for the returned class, model version, and request identifier. Probabilities are model outputs, not calibrated medical certainty.

The Docker image packages the trained pipeline with a locked serving environment and runs as a non-root user. The model is included in the image, so startup does not download data or retrain.

The isolated container was built and tested without host model mounts. Smoke checks verified prediction behavior, invalid-input handling, and the metrics endpoint.



Swagger response from the local API; separate isolated-container verification is recorded in the delivery evidence.

Kubernetes and Operational Monitoring

Stateless replicas, explicit exposure, and per-pod telemetry

The API runs as a small set of stateless Kubernetes replicas behind a local LoadBalancer. Health probes and deployment settings support safe rollout and availability checks.

Prometheus discovers and scrapes API pods individually. Grafana presents request volume, latency, response status, prediction counts, and service availability.

Request logs include operational metadata but omit patient feature values. Monitoring interfaces are accessed locally, and Grafana credentials are stored in a Kubernetes Secret.

A controlled traffic check confirmed that both API pods were monitored and that request metrics were populated. This demonstrates operations monitoring, not model accuracy or drift detection. Local monitoring storage is temporary and is not a production retention setup.



Controlled API traffic, healthy pod monitoring, and verified local LoadBalancer access.

Evidence, Limitations, and Submission

Measured outcomes and an explicit completion boundary

This project demonstrates a reproducible workflow from data preparation through monitored API serving. Results describe a small, historical dataset and should not be treated as performance guarantees for other populations.

The repository includes the source, data instructions, tests, experiment outputs, deployment configuration, screenshots, recording, and report. The report builder checks for required evidence and the requested page count.

The local deployment is intended for an assignment demonstration. Clinical use would require independent validation, clinical review, and appropriate security and operational controls.

The dataset is attributed to Janosi and colleagues and the UCI Machine Learning Repository under CC BY 4.0. Library and platform documentation informed the implementation.

Repository: https://github.com/ayushdev/Heart_Disease_Prediction

Video: MLOps_Recording.mp4

Local API: http://localhost:8000/docs; deployed access: http://localhost:8080/docs after successful Kubernetes verification.

Outstanding: None